

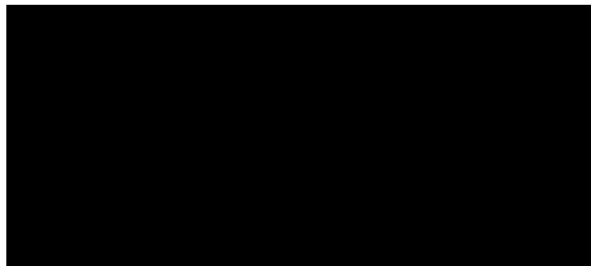
电子科技大学

计算机专业类课程

实验报告

课程名称：计算机网络编程

学 院：计算机学院



指导教师：向渝

日 期： 2025 年 12 月 14 日

电子科技大学

实验报告

实验五

一、实验名称：面向连接的单线程、并发 Echo 服务器研究

二、实验学时：4

三、实验内容和目的：

掌握单线程并发服务端软件的工作原理。

通过研究文件描述符集在交互过程中的变化情况，掌握文件描述符集和 select 系统调用的工作原理

四、实验步骤：

本实验采用 面向连接的 TCP 通信方式，基于 select 系统调用 实现单线程并发 Echo 服务器。服务器在单一进程、单一线程条件下，通过维护文件描述符集合，实现对多个客户端连接的同时监听与服务，从而达到并发处理的目的。

4.1 整体设计思路

服务器程序整体采用“监听 socket + 多客户端 socket + select 轮询”的设计结构，其核心思想如下：

创建一个 TCP 监听 socket，用于接收新的客户端连接请求；

将监听 socket 加入文件描述符集合 allfds；

使用 select 系统调用同时监听监听 socket 与所有已连接客户端 socket 的读事件；

当监听 socket 就绪时，调用 accept 接受新连接，并将新的客户端 socket 加入文件描述符集合；

当某个客户端 socket 就绪时，读取其发送的数据并原样回显（Echo）；
若客户端关闭连接，则移除对应文件描述符并释放资源。

该设计避免了多线程或多进程模型带来的上下文切换开销，体现了 select 在单线程并发服务器中的典型应用模式。

4.2 文件描述符集合的设计与维护

程序中定义了两个文件描述符集合：

allfds：保存当前所有需要监听的文件描述符；

readfds：每次调用 select 前由 allfds 拷贝而来，用于检测读就绪事件。

其中，监听 socket 与所有客户端 socket 均被动态加入或移除 allfds，从而反映服务器当前的连接状态。

fd_set 定义

```
fd_set allfds; //all fds
fd_set readfds; //select
```

FD_ZERO、FD_SET(listenfd, &allfds)

```
//init fd_set
FD_ZERO(&allfds);
FD_SET(listenfd, &allfds);
maxfd = listenfd;
```

readfds = allfds

```
while(1){
    readfds = allfds;
```

4.3 基于 select 的并发处理流程

select 是本实验实现单线程并发的核心。其工作流程如下：

select 阻塞等待，直到至少有一个文件描述符变为就绪状态；

若监听 socket 就绪，说明有新的客户端连接请求；

若客户端 socket 就绪，说明该客户端发送了数据或关闭了连接；

服务器依次判断各就绪描述符并进行相应处理。

通过这种方式，服务器在同一时刻仅使用一个线程，即可实现对多个客户端的并发服务。

```

readfds=allfds,
int ready=select(maxfd+1,&readfds,NULL,NULL,NULL);
if(ready<0){
    perror("select error\n");
    break;
}
//listen socket ready,new connect
if(FD_ISSET(listenfd,&readfds)){
    client_len=sizeof(client_addr);
    connfd=accept(listenfd,(struct sockaddr *)&client_addr,&client_len);
    if(connfd<0){
        perror("accept error\n");
        continue;
    }
    printf("New client connected:%s:%d(fd=%d)
\n",inet_ntoa(client_addr.sin_addr),ntohs(client_addr.sin_port),connfd);

```

4.4 客户端连接管理与 Echo 实现

服务器通过一个整型数组 `clientfds[]` 记录当前所有客户端 socket。每当有新连接建立时，将其存入数组空闲位置；当客户端断开连接时，将对应位置重置为 -1。

Echo 功能的实现流程为：

服务器读取客户端发送的数据；

若读取字节数大于 0，则将数据原样写回客户端；

若读取返回 0 或负值，则判定客户端关闭连接，清理资源。

【此处粘贴代码截图】

内容要求：

```

//tackle connected client data
for(i=0;i<MAX_CLIENTS;i++){
    int sockfd=clientfds[i];
    if(sockfd<0)
        continue;
    if(FD_ISSET(sockfd,&readfds)){
        int n=read(sockfd,buffer,BUFFER_SIZE);
        if(n<=0){
            //close client
            printf("Client fd = %d disconnected\n",sockfd);
            close(sockfd);
            FD_CLR(sockfd,&allfds);
            clientfds[i]=-1;
        }else{
            //echo report
            buffer[n]='\0';
            printf("Received from fd=%d:%s",sockfd,buffer);
            write(sockfd,buffer,n);
        }
        if(--ready<=0)
            break;
    }
}

```

五、实验数据及结果分析：

5.1 多客户端并发连接验证

在实验过程中，通过在多个终端中同时启动客户端程序连接服务器，服务器能够同时接受并维护多个 TCP 连接，且各连接互不影响，验证了 单线程并发模型的正确性。

服务器终端输出能够正确显示每个客户端的连接信息，包括客户端 IP 地址、端口号及对应的文件描述符。

```
yar@2023080910002:~/computer_network$ ./echo_server_select
Echo server started on port 8888...
New client connected:127.0.0.1:47624(fd=4)
New client connected:127.0.0.1:57614(fd=5)
```

5.2 Echo 功能正确性验证

客户端向服务器发送任意字符串数据后，服务器能够即时接收并原样回显至客户端。实验结果表明：数据传输内容未发生丢失或篡改；多个客户端同时发送数据时，服务器均能正确区分并响应；Echo 功能在并发场景下运行稳定。

```
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
hello im 47624
hello im 47624
```

```
Received from fd=4:hello im 47624
```

5.3 客户端断开连接处理验证

当客户端主动关闭连接时，服务器能够正确检测读操作返回值变化，并完成如下操作：输出客户端断开提示信息；关闭对应 socket；从文件描述符集合中移除该描述符。

实验过程中未出现文件描述符泄漏或服务器异常退出现象，表明服务器对连接生命周期的管理是正确的。

客户端退出

```
^]
telnet> quit
Connection closed.
```

服务器端显示 “Client disconnected” 信息

```
Client fd = 4 disconnected
```

5.4 实验结果分析与总结

实验结果表明，基于 `select` 的单线程并发 Echo 服务器能够稳定地处理多个客户端连接请求，并完成数据交互任务。通过文件描述符集合的动态维护，服务器在不引入多线程的情况下实现了并发处理能力，达到了本实验的预期目标。

本实验加深了对 `select` 系统调用机制、文件描述符集合管理以及单线程并发服务器设计思想的理解，为后续学习高并发网络程序设计奠定了基础。